Project - Low Level Design
On

Multi-Agent
Manufacturing System

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	SRASHTI SEN	EN22CS301983
2.	SOUMYA SHRIVASTAVA	EN22CS301978
3.	TANISHKA RAGHUWANSHI	EN22CS3011019
4.	TARUN PATIDAR	EN22CS3011034
5.	VIDHI PRAJAPATI	EN22CS3011076

Group Name: Group 12

Project Number: 12

Industry Mentor Name: Mr. Ashwin

University Mentor Name: Prof. Pradeep Baniya

Academic Year: 2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. References

1. Introduction

With the advancement of **Agentic Artificial Intelligence**, systems are no longer limited to a single monolithic AI model. Instead, intelligent workflows can be created using multiple specialized agents that collaborate to solve complex real-world problems.

This project implements a **Multi-Agent Manufacturing Decision System**, where multiple AI agents work together to perform supplier analysis and generate structured reports.

Unlike traditional systems, this architecture divides responsibilities among agents:

- A **Researcher Agent** gathers structured supplier data
- A **Writer Agent** analyzes and converts data into insights

The system demonstrates how **multi-agent collaboration, task orchestration, and structured communication (JSON)** can be used to solve manufacturing decision-making problems efficiently.

1.1 Scope of the Document

This document provides the Low-Level Design of the **Agentic AI Manufacturing System**. It includes:

- Internal architecture of multi-agent workflow
- Agent design and responsibilities
- Task orchestration mechanism
- Data and process flow
- API and LLM integration
- Design considerations

1.2 Intended Audience

This document is intended for the following stakeholders:

- Software Developers – Implementation reference
-
- System Architects – Design understanding
- Faculty / Evaluators – Project assessment

1.3 System Overview

The Agentic AI system is designed to process user queries related to manufacturing workflows and generate structured, decision-ready outputs using collaborative AI agents.

Unlike traditional AI systems that rely on a single prompt-response mechanism, this system introduces **task decomposition and agent specialization**. The user query is broken down into smaller subtasks, which are then executed sequentially by different agents.

The workflow of the system includes:

1. **Information Retrieval** – Conducted by the Researcher Agent
2. **Data Structuring (JSON)** – Ensures machine-readable output
3. **Analysis & Recommendation** – Performed by the Writer Agent
4. **Final Presentation** – Displayed via Streamlit UI

The system is intentionally **domain-restricted to manufacturing workflows**, which improves response accuracy, reduces ambiguity, and ensures that generated outputs are relevant and reliable.

2. System Design

2.1 Application Design

The system follows a **three-tier architecture** consisting of:

The system follows a **layered architectural pattern**, ensuring separation of concerns and maintainability.

Presentation Layer

The presentation layer is implemented using **Streamlit**, which provides an interactive and lightweight interface for user interaction. It allows users to input queries and view generated outputs in real time.

This layer is responsible only for interaction and does not contain any business logic, ensuring a clean separation between UI and backend processing.

Application Layer

The application layer serves as the core processing unit of the system. It is implemented in Python and handles:

- Agent creation and configuration
- Task definition and assignment
- Workflow orchestration
- Data flow between components

The run() function acts as the central controller, coordinating all system operations from input handling to final output generation.

Agentic AI Layer

This layer represents the intelligence of the system. It consists of multiple AI agents, each with:

- A defined role
- A specific goal
- Contextual backstory

This design enables agents to behave like domain experts, improving the relevance and quality of outputs.

The use of **LLMs as cognitive engines** allows agents to:

- Understand natural language
- Generate structured outputs
- Perform reasoning and comparison

2.2 Process Flow

Workflow begins when the user submits a query through the frontend interface. This query is passed to the backend, where the main orchestration logic is executed.

The system dynamically creates agents and assigns them tasks. The Researcher Agent executes the first task, generating structured data in JSON format. This output is then passed as context to the Writer Agent, which performs analysis and generates a final report.

This step-by-step execution ensures:

- Logical flow of information
- Reduced errors
- Improved traceability of outputs

2.3 Information Flow

User Input

↓

Streamlit UI (app.py)

↓

run() Function

↓

main.py (Agent Orchestration)

↓

Researcher Agent

↓

JSON Output

↓

Writer Agent

↓

Final Report

↓

Frontend Display

2.4 Components Design

1. User Interface Module (app.py)
    - Accepts user input
    - Calls backend
    - Displays output
2. Configuration Module (config.py)
    - Loads API keys
    - Configures LLM
3. Agent Module

Researcher Agent

- Role: Supplier Research Specialist
- Output: Structured JSON

Writer Agent

- Role: Report Generator
- Output: Final formatted report

4. Task Module

Research Task

- Finds supplier details
- Outputs strict JSON

Write Task

- Uses JSON
- Generates insights & recommendations

5. Orchestration Module (main.py)
    - Creates agents
    - Assigns tasks
    - Manages execution using CrewAI
    - Passes context between tasks

2.5 Key Design Considerations

Several factors were considered while designing the system:

Scalability

The system should handle multiple user requests and support future integration of additional AI models.

Performance

Efficient API calls and response handling ensure faster output generation.

Usability

The interface should be simple and easy for users without technical knowledge.

Reliability

Proper error handling ensures the system functions even when APIs fail or network issues occur.

Security

API keys and credentials must be securely stored to prevent unauthorized access.

2.6 API Catalogue

1. Domain Restriction

System only accepts manufacturing-related queries to improve accuracy.

2. Modularity

Each agent has a single responsibility → improves maintainability.

3. Scalability

New agents (e.g., Pricing Agent, Logistics Agent) can be added easily.

4. Structured Communication (JSON)

Ensures:

- Clean data transfer
- No ambiguity

- Better reliability

5. Performance

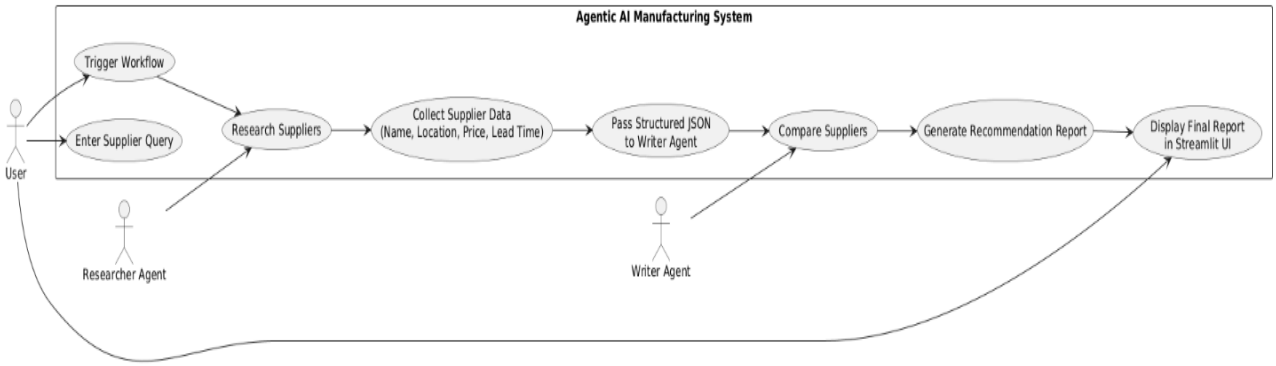
Sequential task execution ensures controlled workflow.

6. Reliability

- Error handling in API calls
- Validation of user input

4.Architecture Diagram

Use Case Diagram



Activity Diagram

5. References

1. Goodfellow, I., Bengio, Y., & Courville, A. – *Deep Learning*

2. OpenAI Documentation – Generative AI APIs
3. Hugging Face Transformers Documentation
4. Research papers on **Multimodal Artificial Intelligence**
5. Generative AI Applications in Manufacturing – IEEE Publications